

A Grammar of Model Parameters for Epidemiological Simulation and Calibration

Vince Buffalo

Institute for Disease Modeling
Gates Foundation
Seattle, WA

December 15, 2025

Introduction

In the last decade, a key innovation in data analytics software has been the development of “grammars” that decouple *what* data analysts want to do from *how* those operations are implemented. Wilkinson’s “grammar of graphics” (Wilkinson 2005) posed creating statistical graphics as not a menagerie of different plots (e.g. boxplots, scatterplots, histograms), but a composable grammar that *maps* data to graphical primitives (e.g. aesthetic attributes, geometries, scales, and facets). This powerful abstraction was the foundation for the pioneering and popular `ggplot2` package in R. Critically, grammars empower users to compose complex plots from this modular, intuitive set of graphical components that directly map the underlying data, rather than requiring users to learn a new interface for each plot, or tweak low-level rendering code.

Similarly, Wickham and François’s `dplyr` package have broadened the idea of grammars to data manipulation (Wickham et al. 2023). Here, the grammar is a set of verbs like `filter`, `select`, `mutate`, and `summarize` that underpin common data manipulation operations. These simple verbs can be composed into powerful data processing *pipelines*, similar to how Unix pipes (McIlroy 2019) can be used to build text-processing programs up from simple Unix data tools. The power of this approach is evidenced by the popularity of Unix tooling and pipes in the processing of large-scale genomics data (Buffalo 2015). Overall, the success of these grammars has been driven by their ability to empower users to reason through complex tasks using a small set of intuitive, composable operations defined by the underlying data grammar.

This paper proposes a *grammar of model parameters* for epidemiological modeling (though the ideas are broadly applicable to other domains). The motivation for this specific grammar comes from the observation that like statistical graphics and data manipulation tasks, many routine tasks during simulation and calibration of epidemiological models can be viewed as compositions of simple operations on model parameter spaces. A concrete grammar can then be the foundation of iterative model building, model fitting, model checking and validation, and model comparison *workflows* (Gelman et al. 2020) that are built using its grammatical primitives. The impetus for a grammar is the realization that many modeling tasks—a large fraction of the modeler’s day-to-day work—can be decomposed into a small set of composable operations.

Partitioning Model Inputs

In computational epidemiological modeling, we typically have one *base model* that is defined by a set of input parameters and settings. In large models, these input parameters and settings can be numerous and complex, ranging from biological parameters (e.g. transmission rates, incubation periods), to demographic parameters (e.g. age distributions, contact matrices), to intervention parameters (e.g. vaccination coverage, vaccine efficacy, school closures), to runtime simulation settings (e.g. scaling factors, time steps, random seeds). The high-dimensional space of all possible input parameters and settings can be not only challenging to fit data to, but can be difficult in practice to work with. For example, when calibrating a model to data, we often only want to vary a small subset of the unknown model parameters, while holding the rest fixed at known or assumed values. Many common software interfaces for specifying and adjusting parameters (e.g. YAML or JSON configuration files, command line arguments, etc.) can become extremely complicated when trying to specify which parameters should be free to vary during calibration. This complexity can lead to errors, confusion, and lost provenance of the origination of fixed model parameters in calibration workflows.

The grammar of model parameters proposed here addresses this challenge by taking an opinionated stance on how all *model inputs* should be partitioned. Specifically, we partition the full set of model inputs into three concrete categories:

1. *Model parameters* ($\mathcal{M}$) that define the “tuneable knobs”. The criteria for inclusion into the set of model parameters is a model input that *could* be varied during a calibration or sensitivity analysis. These do not need to be actively calibrated in a given analysis; through the grammar they can be easily *fixed* to specified values to reduce the dimensionality of the space when desired.
2. *Model configurations* ($\mathcal{C}$) that are runtime implementation choices that are distinguishable from model parameters in that they *will never be varied* during calibration. For example, population size and structure, time step, scaling factors, which outputs to record, filepaths, and scenario-specific switches (e.g. enable school closures, or which vaccine product is used) that are not calibrated.
3. *Base random seed* ($s \in \mathcal{S}$) that is used to reproducibly initialize the pseudo-random number generator (PRNG) used during stochastic simulation. In stochastic simulations, this is a first-class object that is intentionally separated from model configurations because it indexes stochastic replicates and thus must be standardized and easily accessible during calibration and downstream workflows (such as uncertainty quantification).

Mathematically, the full set of model inputs for a stochastic simulation Sim then defined as the triplet

$$\text{Sim} : \mathcal{M} \times \mathcal{C} \times \mathcal{S} \rightarrow \mathcal{Y},$$

which produces output (e.g. observed data, latent state, etc.) $\mathcal{Y}$.

A particular simulation run with model parameters $m \in \mathcal{M}$, configuration $c \in \mathcal{C}$, and seed $s \in \mathcal{S}$ is a mapping $(m, c, s) \mapsto y$.

Scenarios as Patches

In many epidemiological models, researchers wish to compare different scenarios, which are simulations under different fixed parameters or with altered configurations. Given that the simulation algorithm Sim defines a purely functional, deterministic mapping of parameters m , configuration c , and random seed s , we can without loss of generality define a *model scenario* as a deterministic map that *patches parameters and/or configurations* from their default or “baseline” values to a specific scenario. The patch-based design of scenarios enables users to alter the behavior of simulations in a contained, composable, and reproducible way without requiring modifications to the underlying simulation code. This reduces code turnover which could invalidate the reproducibility of past results or introduce bugs.

Before defining formally what scenarios are, we look at two examples of epidemiological scenarios and how they can be framed as scenario patches. First, consider an epidemiological simulation model in which vaccine coverage is varied. Since vaccination coverage is not an unknown knob that would be tuned during a calibration, it would be model configuration. A “baseline” scenario would be a vaccine coverage of 0%, corresponding to no vaccine intervention; alternatively, vaccine interventions could be scaled continuously to varying levels of target coverage. Different intervention efforts (e.g. high, medium, low) then naturally correspond to different patches that alter the target vaccination coverage amount.

Second, consider a scenario that alters a biological epidemiological parameter, such as transmissibility. Suppose a COVID model has been previously calibrated to observed Delta variant wave incidence data, which lead to a β_0 parameter estimate. However, a researcher wishes to see how hospital bed usage would increase under a Omicron-like variant that increases parameter β by a fixed factor such as $1.5\times$ or $2\times$. This more-transmissible strain scenario is then a simple functional patch that takes an existing parameter value and modifies it via some scalar-valued function, e.g. $\beta_0 \mapsto k_\beta \beta$ where k_β is a transmissibility scalar.

Scenario-augmented Simulation

Formally, a scenario is a deterministic map $\sigma : \mathcal{M} \times \mathcal{C} \mapsto \mathcal{M} \times \mathcal{C}$ that patches, or modifies, an existing set of parameters and configuration. One can decompose the patch into parameter-modifying and configuration-modifying components,

$$\sigma(m, c) = (\sigma_{\mathcal{M}}(m, c), \sigma_{\mathcal{C}}(m, c))$$

such that we end up with a scenario patch

$$(m', c') = \sigma(m, c).$$

Critically, a scenario patch does not modify the index set of parameters (no parameter additions or removals); only parameter values can be changed. Parameters and/or configurations can only be equal to their original values, or modified by a deterministic function of (m, c) .

Then, for a simulator $\text{Sim}(m, c, s)$, we can apply a scenario patch σ to the pair (m, c_0) , where c_0 is the baseline configuration. By the decomposability of scenario patches, the simulation routine for a scenario σ is

$$\text{Sim}_\sigma(m, s) := \text{Sim}(m', c', s) = \text{Sim}(\sigma_{\mathcal{M}}(m, c_0), \sigma_{\mathcal{C}}(m, c_0), s).$$

Then, the *baseline scenario* is the *identity patch* $\sigma_{\text{baseline}}(m, c) = (m, c)$ and the baseline model is then $\text{Sim}_{\text{baseline}}(m, s) = \text{Sim}(m, c_0, s)$.

Conceptually, the baseline scenario encodes the data-generating world against which the model is calibrated. Most calibration workflows therefore operate on the baseline scenario (or a small number of closely related variants), treating scenarios themselves as counterfactual modifications to be explored *after* calibration. While nothing in the grammar forbids calibrating under a non-baseline scenario, the intended design is that scenarios primarily encode policy or environment changes, not routine calibration knobs.

Composition of Scenarios

The patch-based design of scenarios enables users to compose multiple scenarios in a way with intuitive semantics and expected behavior. Since scenarios can neither modify the model parameter space $\mathcal{M}$ nor the configuration space $\mathcal{C}$, scenarios are closed under composition. Given σ_1 and σ_2 , their composition $(\sigma_2 \circ \sigma_1)(m, c) := \sigma_2(\sigma_1(m, c))$. Furthermore, scenarios are associative,

$$\sigma_3 \circ (\sigma_2 \circ \sigma_1) = (\sigma_3 \circ \sigma_2) \circ \sigma_1$$

which implies a simulation of composed scenario layers the scenarios as a user would expect,

$$\text{Sim}_{\sigma_2 \circ \sigma_1}(m, s) = \text{Sim}((\sigma_2 \circ \sigma_1)(m, c_0), s).$$

To illustrate a practical benefit of the composition of multiple scenarios, consider how a modeler would implement and combine a “variant” scenario and a school closure scenario. The variant scenario is defined as $\sigma_{\text{variant}}(m, c) = (m', c)$ that increases a transmissibility model parameter by a fixed factor $m'_\beta = k_\beta m_\beta$, and the school closure scenario is defined as $\sigma_{\text{schools}}(m, c) = (m, c')$ where c' alters the configuration to close schools during certain time periods. In this example, a user can easily run simulations under either or both scenarios. Additionally, since these particular scenario patches are modifying disjoint parts of $(\mathcal{M}, \mathcal{C})$, these particular scenarios are commutative: $\sigma_{\text{variant}} \circ \sigma_{\text{schools}} = \sigma_{\text{schools}} \circ \sigma_{\text{variant}}$. However, in general scenarios are *not* commutative.

To illustrate a case where scenarios are not commutative, consider a pair of scenarios that both modify vaccine coverage. Let $\sigma_{\text{cov-base}}$ be a scenario that pins coverage to a fixed target level, e.g. $\sigma_{\text{cov-base}}(m, c) = (m', c)$ with $m'_{\text{cov}} = 0.7$ and $m'_n = m_n$ for all other parameters n . Let $\sigma_{\text{cov-scale}}$ be a scenario that scales whatever coverage is present by a factor $k > 1$, capped at 1: $\sigma_{\text{cov-scale}}(m, c) = (m'', c)$ with $m''_{\text{cov}} = \min(1, k m_{\text{cov}})$ and $m''_n = m_n$ for all other n . For the same baseline (m, c) , the composed scenario $\sigma_{\text{cov-scale}} \circ \sigma_{\text{cov-base}}$ yields coverage $m'''_{\text{cov}} = \min(1, k \cdot 0.7)$, whereas $\sigma_{\text{cov-base}} \circ \sigma_{\text{cov-scale}}$ always yields $m'''_{\text{cov}} = 0.7$, since the base-coverage scenario overwrites any previous scaling. Both compositions are valid patches $\mathcal{M} \times \mathcal{C} \rightarrow \mathcal{M} \times \mathcal{C}$, but they encode different semantics: order matters whenever scenarios patch overlapping components of $(\mathcal{M}, \mathcal{C})$.

Importantly, it is possible to implement a subset of scenario patching operations such that the non-commutativity of scenarios can be identified. This would enable software to warn when one scenario updates or overwrites a modification of a previous scenario patch. For example, if scenarios are implemented as structured modifications (as in Listing 1), rather than arbitrary functions in code, they can be programmatically checked for non-commutativity.

```

1  from dataclasses import dataclass
2  from typing import Literal
3
4  @dataclass(frozen=True)
5  class ParamPatch:
6      kind: Literal["scale", "add"] # commutative operations
7      param: str
8      value: float
9
10 @dataclass(frozen=True)
11 class ScenarioSpec:
12     param_patches: list[ParamPatch]
13     config_patches: list[ConfigPatch]

```

Listing 1: Example Python implementation of scenario patch specifications using dataclasses. When scenarios are represented as static, structured patches rather than arbitrary functions, their *write sets* and *read sets* (which parameters and configuration fields they modify or depend on) can be computed directly. If the write sets of two scenarios are disjoint, their composition is always commutative. If the write sets overlap, commutativity holds if and only if the algebra of the underlying update operations on the overlapping fields is itself commutative.

The Grammar of Model Parameters

Epidemiological models can have a slew of parameters that may be subject to calibration to observed data, creating a complex high-dimensional *model parameter space* $\mathcal{M}$. Many routine tasks a modeler does during initial model exploration, calibration to observed data, post-calibration diagnostics, model validation, prediction, and sensitivity analyses are operations on model parameters that an expressive grammar should make trivial to do. In order to formalize these common operations, we must first mathematically define what these common operations do.

The practical benefit of a grammar of model parameters is maximized when the implementation of the grammar is external to, or “sits above”, the underlying simulation algorithm $\text{Sim}(m, c, s)$. This allows the user to interact with the underlying model’s parameters, doing routine operations like exploring reduced parameter spaces by fixing or conditioning on values, transforming parameters, reparameterizing models (defining a new, reduced parameter space by e.g. creating a compound parameter $(m_1, m_2) \mapsto m_1 \cdot m_2$) via a *higher-level abstraction*. A major benefit of the decoupling between simulation algorithm $\text{Sim}(m, c, s)$ and the parameter grammar is that the UX of abstractions for common model parameter manipulations can be improved without altering the underlying Sim code, which would unnecessarily invalidate model runs and their results. Moreover, decoupling the model parameter grammar from the underlying simulation algorithm means researchers can effortlessly switch between simulation frameworks without the cognitive burden of having to learn a framework’s parameter specification.

Model Parameter Space $\mathcal{M}$

As described earlier, the sum total of all model input into a simulation algorithm $\text{Sim}(m, c, s)$ can be partitioned into model parameters $m \in \mathcal{M}$, model configuration $c \in \mathcal{C}$, and a simulation seed $s \in \mathcal{S}$. Since an external implementation of the model parameter grammar sits above a model’s core $\text{Sim}(m, c, s)$ interface, the underlying simulation routine must expose all model parameters that may

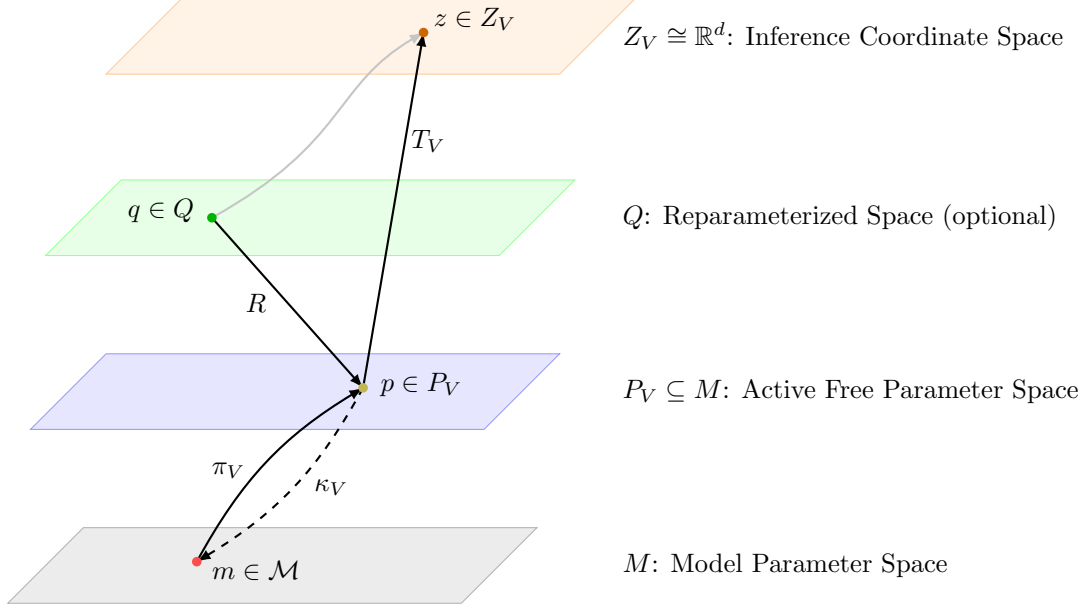


Figure 1: An illustration of the model parameter space $\mathcal{M}$ and all derived spaces that a user may encounter via operations in the parameter grammar.

be subject to calibration, if not all model parameters. This is expanding the model parameter space $\mathcal{M}$ to its full dimensionality, unconstraining the user to do all parameter manipulations without having to change the underlying simulation code.

Formally, for a finite index set S of parameter names (e.g. `beta`, `p_vax`), the *model parameter space* $\mathcal{M}$ is defined as the product space

$$\mathcal{M} := \prod_{n \in S} \text{Dom}(n)$$

where $\text{Dom}(n)$ is the one-dimensional domain of parameter n in the index set S . Then, a point in model parameter space $m \in \mathcal{M}$ is a complete assignment of the form

$$m = (m_n)_{n \in S}, \quad m_n \in \text{Dom}(n).$$

which can be seen in lowest space in Figure 1.

Configuration Space $\mathcal{C}$

In the partitioning of model inputs, the configuration space $\mathcal{C}$ collects all runtime and structural settings that are *not* subject to calibration: population size and structure, time step, scaling factors, output specifications, filepaths, and scenario-specific switches that are treated as fixed for a given analysis.

Formally, we can regard $\mathcal{C}$ as an arbitrary product space

$$\mathcal{C} := \prod_{k \in K} \text{ConfDom}(k),$$

where K indexes configuration fields and each $\text{ConfDom}(k)$ is an appropriate domain (continuous, discrete, categorical, etc.). The grammar itself does not require any particular typing discipline for $\mathcal{C}$; it only assumes that, for a given model, there is a well-defined configuration space and a baseline configuration $c_0 \in \mathcal{C}$.

Implementation-wise, one may choose to mirror the treatment of $\mathcal{M}$ with a strongly typed `ConfigurationSpace/ConfigurationSet` or to use a more lightweight dictionary-based representation. The key invariant is that scenarios act as patches

$$\sigma : \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{M} \times \mathcal{C},$$

regardless of how $\mathcal{C}$ is represented in code.

Active Free Parameter Space P_V

When the underlying simulation algorithm $\text{Sim}(m, c, s)$ exposes a possibly high-dimensional model parameter space $\mathcal{M}$, users often need to run simulations or calibrations that condition on, or fix, certain parameters. This may be for practical reasons (e.g. the dimensionality of $\mathcal{M}$ exposed by $\text{Sim}(m, c, s)$ is large, but most parameters are known with sufficient accuracy they can be treated as fixed), or during iterative model building as part of testing calibration or doing sensitivity analyses. Parameter views are useful not only for calibration but also for parameter sweeps, sensitivity analyses, and any workflow requiring reduced dimensionality.

When a user fixes one or more parameters to particular values, they are in essence defining a *parameter view* V which is the pair $V = (F, \text{fixed}_V)$, where F is the subset of *fixed parameters* $F \subseteq S$ that are fixed, and the mapping $\text{fixed}_V : F \rightarrow \mathbb{R}$ that gives the fixed value for each fixed parameter $n \in F$ with $\text{fixed}_V(n) \in \text{Dom}(n)$.

Then, the parameter view V , induces a corresponding *free set* $\text{free}(V) := S \setminus F$, and an *active free parameter space* P_V ,

$$P_V := \prod_{n \in \text{free}(V)} \text{Dom}(n) \subseteq \mathcal{M}$$

where $P_V = \mathcal{M}$ iff $F = \emptyset$.

A non-null fix set induces two canonical maps. First, fixing parameters drops the fixed coordinates and keeps only the free ones. This creates a projection π_V between the model parameter space $\mathcal{M}$ and the parameter view space P_V ,

$$\pi_V : \mathcal{M} \rightarrow P_V$$

that drops the fixed coordinates, keeping only the free ones, i.e.

$$\pi_V((m_n)_{n \in S}) = (m_n)_{n \in \text{free}(V)}.$$

Intuitively, this projection can be thought of as the reduced “active set” of free parameters, used for calibration or to generate simulation parameter grids over the subset of free parameters.

Second, we need an embedding or “fill map”, which is the inverse operation of the projection π_V . In practice this is necessary because the simulation algorithm $\text{Sim}(m, c, s)$ cannot run unless $m \in \mathcal{M}$; a point in the free active parameter space $p \in P_V$ must be embedded back into $\mathcal{M}$ by

filling in the fixed values that created the reduced space P_V . We denote this embedding or “fill map” κ_V ,

$$\kappa_V : P_V \rightarrow \mathcal{M}$$

and it fills back in the fixed parameter values from the parameter view V :

$$(p_n)_{n \in \text{free}(V)} \mapsto (m'_n)_{n \in S} \quad \text{with} \quad m'_n = \begin{cases} p_n, & n \in \text{free}(V), \\ \text{fixed}_V(n), & n \in F. \end{cases}$$

Then, on the subset of $\mathcal{M}$ consistent with the fixed parameters in V (i.e. $m_n = \text{fixed}_V(n)$ for all $n \in F$), the composition of the projection map and embedding map is the identity,

$$\kappa_V \circ \pi_V = \text{id}.$$

Importantly, parameter views naturally define a new simulator interface on P_V ,

$$\text{Sim}_{\sigma,V}(p, s) = \text{Sim}_{\sigma}(\kappa_V(p), s).$$

Reparameterization Space Q (Optional)

In many scientific models, it is useful to define new parameters as functions of existing parameters. For example, in epidemiological models, the basic reproduction number R_0 is often expressed in terms of transmission and duration parameters, such as β and γ . To capture such changes of coordinates, we introduce an (optional) *reparameterized space* Q together with a map $R : Q \rightarrow P_V$. In the simplest case R is a bijection (i.e. one-to-one and onto, or a pure reparameterization with $\dim Q = \dim P_V$); more generally, R may be an injective map (one-to-one) that encodes constraints between free parameters, so that $\dim Q \leq \dim P_V$, as would be the case when defining a compound parameter.

In many applications, no explicit reparameterization space is needed, and one simply works with $Q = P_V$ and $R = \text{id}$. The Q -layer is therefore an optional, advanced extension for cases where compound parameters or lower-dimensional manifolds provide substantial practical benefit (e.g. for identifiability or interpretability).

For a *pure reparameterization*, the map R is a bijection such that for each $q \in Q$, there is a unique $p \in P_V$ with $p = R(q)$. Then, the inverse map $R^{-1} : P_V \rightarrow Q$ exists, and the induced simulator in Q coordinates is

$$\text{Sim}_{\sigma,V,R}(q, s) := \text{Sim}_{\sigma}(\kappa_V(R(q)), s), \quad q \in Q.$$

Because R is a bijection, we can equivalently express the simulator in P_V : for any $p \in P_V$ with $q = R^{-1}(p)$,

$$\text{Sim}_{\sigma,V}(p, s) = \text{Sim}_{\sigma,V,R}(R^{-1}(p), s).$$

For example, for a simple SEIR model, suppose instead of transmission parameter β and recovery parameter γ , we wish to reparameterize the model in terms of $R_0 = \beta/\gamma$ and the infectious period $D = 1/\gamma$. This reparameterization defines a new coordinate space

$$Q := \{(R_0, D) \in (0, \infty)^2\},$$

and a reparameterization map

$$R : Q \rightarrow P_V, \quad (R_0, D) \mapsto \left(\beta = \frac{R_0}{D}, \gamma = \frac{1}{D} \right).$$

Moreover, this is a bijection with inverse map

$$R^{-1} : P_V \rightarrow Q, \quad (\beta, \gamma) \mapsto \left(R_0 = \frac{\beta}{\gamma}, D = \frac{1}{\gamma} \right).$$

In other cases (such as reparameterizations that define compound parameters or constraints), the map $R : Q \rightarrow P_V$ is injective but not necessarily surjective, so that $\dim Q \leq \dim P_V$. In this case, the inverse map $R^{-1} : P_V \rightarrow Q$ is not defined on all of P_V , but only on the image of R . Then, this encodes constraints between the free parameters; after such a *reduced or constrained reparameterization*, the simulator is exploring only a subset $R(Q) \subseteq P_V$ of the parameter view space. The induced simulator is still the exact same as Equation (), but in the reduced case, only $p \in R(Q)$ are reachable; the inference space is restricted to a lower-dimension manifold embedded in P_V .

As an example of a reduced reparameterization, consider a model with two free parameters $(m_{\text{adult}}, m_{\text{child}}) = c \cdot (r_{\text{adult}}, r_{\text{child}})$ for $c > 0$, where r_{adult} and r_{child} are fixed from some external or previous analysis. Here, the reparameterization is then $R(c) = (m_{\text{adult}}, m_{\text{child}}) = (cr_{\text{adult}}, cr_{\text{child}}) \in P_V$. Now, $\dim Q = 1 < 2 = \dim P_V$, and calibration is reduced to a one-dimensional manifold in P_V defined by the line $R(Q) = \{c(r_{\text{adult}}, r_{\text{child}}) : c > 0\} \subset P_V$.

Per-Parameter Transformations T_V

Often during model calibration, it is useful to transform parameters by some invertible function (e.g. log, logit, etc.) to make the parameter space more amenable to optimization or sampling algorithms. For example, many parameters may be constrained to be positive (e.g. rates, counts), vary over several orders of magnitude, or constrained to the unit interval (e.g. probabilities, proportions). Transforming these constrained parameters to an unconstrained space (e.g. via log or logit) can improve the performance and stability of calibrations. Typically, a user must change the underlying model code itself so that a model parameter itself $m \in \mathcal{M}$ is transformed before being used in the underlying simulation algorithm. The downside of this approach is that it (1) alters the underlying simulation code, which may invalidate past results or introduce bugs, and (2) is a more time-consuming change to code that makes quick and simple exploratory transformations more difficult.

A grammar of model parameters should make it trivial to apply per-parameter transformations without changing the underlying simulation code. The map $T_V : P_V \rightarrow Z_V$ is defined at the level of a particular view V : for each free coordinate $n \in \text{free}(V)$ in the active free parameter space, a user can optionally choose a scalar, invertible transformation t_n ,

$$t_n : \text{Dom}(n) \rightarrow \mathcal{Z}_n$$

where $\mathcal{Z}_n = \mathbb{R}$ is the transformed domain of parameter n . In an implementation, these transform choices may be attached directly to parameter specifications (providing a default) or supplied at calibration time (overriding defaults). What matters for the grammar is that, for any fixed view V , there is a well-defined product transform T_V and inverse T_V^{-1} ; the mechanism by which those choices are recorded is an engineering decision. In practice, it is often advantageous to keep the

choice of transform at the level of the coordinate system or view, so that different analyses over the same model can use different transforms without modifying the model implementation. For example, some common transformations are

$$\begin{aligned} t_n(x) &= \log(x), & \text{if } \text{Dom}(n) = (0, \infty) \\ t_n(x) &= \log \frac{x}{1-x}, & \text{if } \text{Dom}(n) = (0, 1) \end{aligned}$$

From the set of free parameters and their possible transformations, we then define the *inference coordinate space*

$$Z_V := \prod_{n \in \text{free}(V)} \mathcal{Z}_n,$$

so that each point $z \in Z_V$ is a vector of transformed coordinates $z = (z_n)_{n \in \text{free}(V)}$. The per-parameter transforms induce a product map

$$T_V : P_V \rightarrow Z_V,$$

defined componentwise by

$$T_V((p_n)_{n \in \text{free}(V)}) := (t_n(p_n))_{n \in \text{free}(V)}.$$

Since each t_n is assumed to be invertible on $\text{Dom}(n)$, we obtain an inverse product map

$$T_V^{-1} : Z_V \rightarrow P_V, \quad T_V^{-1}((z_n)_{n \in \text{free}(V)}) := (t_n^{-1}(z_n))_{n \in \text{free}(V)}.$$

Crucially, T_V and T_V^{-1} live entirely in the parameter grammar layer: they remap coordinates on the active free parameter space P_V without requiring any changes to the underlying simulation algorithm $\text{Sim}(m, c, s)$. This allows optimization or sampling algorithms to operate in a numerically convenient space Z_V (typically $\mathbb{R}^d$ with $d = |\text{free}(V)|$), while preserving the scientific interpretation of parameters in $\mathcal{M}$.

Inference Coordinate Space Z_V

From the perspective of an inference algorithm (optimizer, MCMC sampler, SMC, etc.), the primary interface is a function (often a closure around observed data) that accepts a vector in $\mathbb{R}^d$ and a seed, and returns a simulated output. The composition of the maps introduced so far provides exactly such an interface that takes in a point in the inference coordinate $z \in Z_V$ and maps it back to the full model parameter space $\mathcal{M}$ for use in the underlying simulation algorithm $\text{Sim}(m, c, s)$.

In particular, for a point $z \in Z_V$, the parameter grammar maps it through the layers, first recovering a point in the active free parameter space via

$$p = T_V^{-1}(z) \in P_V,$$

then embed this back into the full model parameter space via the fill map

$$m = \kappa_V(p) \in \mathcal{M},$$

and finally apply the scenario patch σ and simulator. This defines the Z_V -parameterized simulator

$$\text{Sim}_{\sigma,V,T}(z, s) := \text{Sim}_{\sigma}(\kappa_V(T_V^{-1}(z)), s), \quad z \in Z_V.$$

Importantly, this simulator interface allows inference algorithms to operate in the inference coordinate space $Z_V \cong \mathbb{R}^d$ without any changes to the underlying simulation algorithm.

When a reparameterization $R : Q \rightarrow P_V$ is also used, the natural composite map to inference coordinates is

$$Q \xrightarrow{R} P_V \xrightarrow{T_V} Z_V.$$

In this case, the corresponding simulator parameterized by $q \in Q$ is

$$\text{Sim}_{\sigma,V,R,T}(q, s) := \text{Sim}_{\sigma}(\kappa_V(R(q)), s) = \text{Sim}_{\sigma,V,T}(T_V(R(q)), s), \quad q \in Q.$$

The latter identity emphasizes that the choice between calibrating in P_V , Q , or Z_V is purely a matter of which coordinate system is most convenient for the inference algorithm. The underlying simulator $\text{Sim}(m, c, s)$ and the scientific interpretation of parameters in $\mathcal{M}$ remain unchanged.

Coordinate Systems

From an implementation perspective, it is useful to package a view V , transforms T_V , and optional reparameterization R into a single *coordinate system* object. This coordinate system encodes:

- Which parameters are fixed vs. free (the view),
- How free parameters are transformed for inference ($T_V : P_V \rightarrow Z_V$),
- Optionally, how a reparameterized space Q maps to P_V (via R).

The downward map from user coordinates to $\mathcal{M}$ is then:

$$z \xrightarrow{T_V^{-1}} P_V \xrightarrow{\kappa_V} \mathcal{M}$$

or, with reparameterization, a coordinate system can be viewed as first inverting transforms to some underlying parameter space Θ (either P_V or Q), then mapping $\Theta \rightarrow P_V$ (via R if $\Theta = Q$), and finally $\kappa_V : P_V \rightarrow \mathcal{M}$. In practice, the transform map T is attached to whichever logical parameter coordinates a coordinate system exposes (either P_V or Q); for concreteness we mostly describe transforms on P_V .

This encapsulation allows implementation of the grammatical operations as transformations on coordinate systems, with the underlying simulator $\text{Sim}(m, c, s)$ remaining unchanged.

Chains of Maps

It is helpful to summarize the grammar in terms of the maps between the spaces introduced above. From the perspective of parameter manipulation and inference, there are two complementary chains, with an optional reparameterization space Q .

By a *simulator interface* we mean any function

$$f : Z \times \mathcal{S} \rightarrow \mathcal{Y}$$

obtained by composing the maps of this grammar, where Z is some coordinate space (e.g., $\mathcal{M}$, P_V , Q , or Z_V) and $\mathcal{S}$ is the seed space. A key engineering invariant suggested by this grammar is that each grammatical operation (fixing parameters via a view, applying a reparameterization, choosing transforms, or composing scenarios) can be implemented as a mapping that takes one simulator interface and returns another simulator interface of the same general shape. Concretely, each layer wraps a function

$$f : \Theta \times \mathcal{S} \rightarrow \mathcal{Y}$$

into a new function with a different parameter space Θ' , without ever modifying the underlying $\text{Sim}(m, c, s)$ implementation.

Upward chains (to inference coordinates). Without an explicit reparameterization, we move from model parameters to inference coordinates via

$$\mathcal{M} \xrightarrow{\pi_V} P_V \xrightarrow{T_V} Z_V.$$

Here π_V projects away the fixed coordinates, and T_V applies the (per-coordinate) transforms t_n to obtain inference coordinates $z \in Z_V$.

When a reparameterization $R : Q \rightarrow P_V$ is used, the upward chain becomes

$$Q \xrightarrow{R} P_V \xrightarrow{T_V} Z_V.$$

In this case Q is simply an alternative coordinate system for the same free directions encoded in P_V (pure reparameterization) or for a constrained submanifold $R(Q) \subseteq P_V$ (reduced reparameterization).

Downward chains (to simulator output). Starting from an inference coordinate $z \in Z_V$ and a seed s , the grammar maps back to simulator output by

$$Z_V \xrightarrow{T_V^{-1}} P_V \xrightarrow{\kappa_V} \mathcal{M} \xrightarrow{\sigma} \mathcal{M} \times \mathcal{C} \xrightarrow{\text{Sim}(\cdot, \cdot, s)} \mathcal{Y}.$$

Here T_V^{-1} inverts the per-parameter transforms; κ_V fills in the fixed coordinates to recover a full parameter vector $m \in \mathcal{M}$; σ applies any scenario patch to (m, c_0) ; and Sim runs the underlying simulator.

If we instead start from a point $q \in Q$, the downward chain is

$$Q \xrightarrow{R} P_V \xrightarrow{\kappa_V} \mathcal{M} \xrightarrow{\sigma} \mathcal{M} \times \mathcal{C} \xrightarrow{\text{Sim}(\cdot, \cdot, s)} \mathcal{Y}.$$

Induced simulator interfaces. Composing the maps in these chains defines several equivalent simulator interfaces:

$$\begin{aligned} \text{Sim}_{\sigma, V, T}(z, s) &:= \text{Sim}_{\sigma}(\kappa_V(T_V^{-1}(z)), s), & z \in Z_V, \\ \text{Sim}_{\sigma, V, R}(q, s) &:= \text{Sim}_{\sigma}(\kappa_V(R(q)), s), & q \in Q. \end{aligned}$$

Because R and T_V are purely grammatical maps, the choice of calibrating in P_V , Q , or Z_V is purely a matter of which coordinate system is most convenient for the inference algorithm; the underlying simulator $\text{Sim}(m, c, s)$ and the scientific meaning of parameters in $\mathcal{M}$ remain unchanged.

Each arrow in the chains above denotes one of these deterministic maps (projections, embeddings, reparameterizations, or per-parameter transforms), and the overall simulator interface is obtained by composing them.

Simulator Interfaces and Composition

A central invariant of the grammar is that every operation ultimately induces a *simulator interface* of the form

$$\text{Sim}^* : X \times \mathcal{S} \rightarrow \mathcal{Y},$$

where X is some parameter coordinate space (e.g. $\mathcal{M}$, P_V , Q , or Z_V) and $\mathcal{S}$ is the seed space. From the perspective of an inference algorithm or workflow engine, what matters is that there is always a function that accepts a parameter point and a seed and returns simulation outputs; the grammar specifies how this interface is constructed.

These distinct mathematical notations (Sim_σ , $\text{Sim}_{\sigma,V}$, $\text{Sim}_{\sigma,V,T}$, etc.) describe how the grammatical maps compose to form different simulator interfaces, but a concrete implementation need not introduce a distinct class for each notation. Rather, all of these can be realized through a single abstraction (e.g., a wrapper object) parameterized by different coordinate system configurations. The important property is *closure*: starting from a simulator interface and applying any grammatical operation yields another simulator interface with the same call signature $(z, \text{seed}) \mapsto \text{outputs}$ but different parameter interpretation.

Starting from the base simulator

$$\text{Sim} : \mathcal{M} \times \mathcal{C} \times \mathcal{S} \rightarrow \mathcal{Y},$$

each grammatical operation produces a new simulator interface:

- A *parameter view* V induces

$$\text{Sim}_{\sigma,V}(p, s) := \text{Sim}_\sigma(\kappa_V(p), s), \quad p \in P_V,$$

which accepts only the free coordinates $p \in P_V$.

- A *reparameterization* $R : Q \rightarrow P_V$ induces

$$\text{Sim}_{\sigma,V,R}(q, s) := \text{Sim}_\sigma(\kappa_V(R(q)), s), \quad q \in Q.$$

- A system of *per-parameter transforms* $T_V : P_V \rightarrow Z_V$ induces the Z_V -parameterized simulator

$$\text{Sim}_{\sigma,V,T}(z, s) := \text{Sim}_\sigma(\kappa_V(T_V^{-1}(z)), s), \quad z \in Z_V.$$

- A *scenario patch* σ modifies $(\mathcal{M}, \mathcal{C})$ but preserves the output space $\mathcal{Y}$, so that every scenario also gives a simulator of the same shape $X \times \mathcal{S} \rightarrow \mathcal{Y}$.

If a reparameterization and transforms are both used, the full downward chain from inference coordinates to simulator output is

$$Z_V \xrightarrow{T_V^{-1}} P_V \xrightarrow{\kappa_V} \mathcal{M} \xrightarrow{\sigma} \mathcal{M} \times \mathcal{C} \xrightarrow{\text{Sim}(\cdot, \cdot, s)} \mathcal{Y}.$$

The important design property is that each grammatical operation is *closed* on the space of simulator interfaces: starting from a base simulator, applying a view, a reparameterization, a transform, or a scenario yields another simulator of the same overall type “parameters $\times$ seed $\rightarrow$ outputs”. This is analogous to how data manipulation grammars (such as `dplyr`) use verbs that map a data frame to a data frame, enabling pipeline composition. Here, the primitives of the parameter grammar compose to produce increasingly specialized simulator interfaces without ever modifying the underlying simulation code.

Example Python Interfaces

This section sketches what a Python API inspired by the grammar might look like from a user’s perspective. The goal is to keep the full formalism $(\mathcal{M}, P_V, Q, Z_V)$ internal while exposing a small set of intuitive objects and methods.

At a high level, such a library would expose four main concepts:

- **Model:** an object that encapsulates the simulation algorithm `Sim`, its parameter space, and available scenarios.
- **Parameter space and sets:** a `ParameterSpace` describing all model parameters $\mathcal{M}$, and `ParameterSet` representing individual points $m \in \mathcal{M}$.
- **Coordinate systems:** a `CoordinateSystem` that packages a `ParameterView` (which parameters are fixed vs. free, inducing P_V), transforms for free parameters (inducing Z_V), and optional reparameterization.
- **Packaged simulator interface:** a `ModelSimulator` that wraps a model, scenario, and coordinate system into a single callable $Z \times \mathcal{S} \rightarrow \mathcal{Y}$ suitable for calibration, parameter sweeps, or scenario exploration.

Defining a Model and its Parameter Space

A model author first defines the parameter space and a baseline parameter set:

```

space = ParameterSpace([
    ParameterSpec("beta", lower=0.01, upper=2.0, doc="Transmission rate"),
    ParameterSpec("gamma", lower=0.05, upper=1.0, doc="Recovery rate"),
    ParameterSpec("p_vax", lower=0.0, upper=1.0, doc="Baseline coverage"),
    ParameterSpec("waning_half_life", lower=30.0, upper=3650.0, doc="Days"),
])

baseline = ParameterSet.new(
    space,
    beta=0.35,
    gamma=0.2,
    p_vax=0.8,
    waning_half_life=365.0,
)

model = MySEIRModel(space=space, base_config={"target_coverage": 0.8})

```

Listing 2: Defining a parameter space $\mathcal{M}$, a baseline parameter set, and constructing a model object. The model exposes `simulate(params, seed)` and `simulate_scenario(name, params, seed)` methods as the core engine on which the parameter grammar is built.

The model provides the base simulator $\text{Sim} : \mathcal{M} \times \mathcal{C} \times \mathcal{S} \rightarrow \mathcal{Y}$ and scenario-augmented variants Sim_σ . These methods enforce that *only* complete `ParameterSet` instances are accepted.

```

@dataclass(frozen=True)
class CoordinateSystem:
    """Coordinate system over model parameters.

    Encodes:
    - a view  $V$  (fixed/free split),
    - per-free-parameter transforms  $T_V$ ,
    - optional reparameterization  $R: Q \rightarrow P_V$ .

    Provides the downward map from user coordinates to  $M$ -space.
    """
    view: ParameterView
    transforms: Mapping[str, Transform] = field(default_factory=dict)
    reparam: Optional[Reparameterization] = None

    def to_M(self, z: np.ndarray) -> ParameterSet:
        """Downward map: user coordinates  $\rightarrow M$  space.

        Implements the chain:
         $Z \rightarrow P_V \rightarrow M$  (or  $Z \rightarrow Q \rightarrow P_V \rightarrow M$  with reparameterization)
        via inverse transforms and view embedding.
        """
        ...

```

Listing 3: The `CoordinateSystem` class packages view, transforms, and optional reparameterization into a single object that converts user coordinates to model parameter space $\mathcal{M}$.

Declaring Views and Scenarios

When working with models, users rarely want to vary all parameters—whether for calibration, sweeps, or scenario exploration. Instead, they fix some and leave others free. This corresponds to selecting a view V and inducing P_V :

```
# Fix p_vax and waning_half_life, leaving beta and gamma free
view = ParameterView.from_fixed(
    space,
    p_vax=0.8,
    waning_half_life=365.0,
)
print(view.free)  # ('beta', 'gamma')
```

Listing 4: A `ParameterView` defines fixed vs. free parameters (inducing P_V) for parameter sweeps, calibration, or sensitivity analysis.

Scenarios are defined as structured patches, following the “scenarios as patches” section. In code, they are concrete objects that the model registers under names:

```
# Example: a policy scenario that increases coverage and adjusts configuration
high_cov = ScenarioSpec(
    name="high_coverage",
    doc="Increase routine vaccination coverage",
    param_patches=[...],  # optional parameter-level patches
    config_patches=[...], # configuration-level patches
)

# Scenarios are registered with the model at definition time, e.g., via decorators
# or other framework-specific mechanisms. The key is that each scenario has a
# stable name that can be referenced when building simulator interfaces.
```

Listing 5: A `ScenarioSpec` object encapsulates a patch $\sigma : \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{M} \times \mathcal{C}$.

By default, calibration is performed against a designated baseline scenario (intended to match the observation process). Policy or counterfactual scenarios (e.g. "high_coverage") are primarily used *after* calibration, by running the calibrated model under different patches.

Transforms and Inference Coordinates

To make calibration numerically stable, free parameters can be transformed to unconstrained coordinates. Transforms are chosen at the calibration layer, rather than being hard-wired into the parameter space. A simple interface might accept a mapping from parameter names to transform kinds:


```

# Specify transforms for this particular calibration
transform_spec = {
    "beta": "log",      # (0, inf) → R
    "gamma": "log",     # (0, inf) → R
    # p_vax and waning_half_life use identity by default
}

```

Listing 6: Transform specification for a particular coordinate system. The library constructs $T_V : P_V \rightarrow Z_V$ from these hints. Different coordinate systems over the same model can use different transform specifications.

Internally, these hints are resolved into concrete `Transform` objects (e.g. log, logit, or identity) and assembled into the product map $T_V : P_V \rightarrow Z_V$ and its inverse $T_V^{-1} : Z_V \rightarrow P_V$. Different calibration tasks on the same model can use different transform specifications without changing the underlying model or parameter space.

A Fluent Calibration Interface

Putting these pieces together, a user-facing calibration interface can present a small, fluent surface API while the grammar remains internal. The key abstraction is a packaged simulator that wraps a model and coordinate system:

```

@dataclass(frozen=True)
class ModelSimulator:
    """Packaged simulator wrapping a BaseModel and a coordinate system.

    Implements the simulator interface: callable with (z, seed) → outputs.
    """
    model: BaseModel
    scenario: str
    coords: CoordinateSystem

    @property
    def dim(self) -> int:
        return self.coords.dim

    @property
    def names(self) -> Tuple[str, ...]:
        return self.coords.param_names

    def __call__(self, z: np.ndarray, seed: int) -> Dict[str, pl.DataFrame]:
        """Execute simulation in coordinate system Z.

        Downward chain: Z → M → Y via coordinate system and base model.
        """
        m = self.coords.to_M(z) # user coords → parameter space
        return self.model.simulate_scenario(self.scenario, m, seed)

```

Listing 7: The `ModelSimulator` class wraps a `BaseModel` and `CoordinateSystem` into a single callable interface $Z \times \mathcal{S} \rightarrow \mathcal{Y}$. Every grammatical operation constructs a new `ModelSimulator` with a modified coordinate system.

This packaged simulator is then constructed via a fluent builder:

```
# Start from BaseModel, build a packaged simulator via fluent API
sim = (
    model
    .as_sim("baseline")           # choose scenario (e.g. "baseline")
    .fix(p_vax=0.8, waning_half_life=365) # narrow view (define V)
    .with_transforms(beta="log", gamma="log") # choose T_V
    .build() # returns ModelSimulator
)

# sim: ModelSimulator with __call__(z: ndarray, seed: int) → outputs
print(sim.names)    # ("beta", "gamma")
print(sim.dim)      # 2

# Use in optimization
def objective(z):
    outputs = sim(z, seed=42) # z is a 2D vector in calibration coordinates
    return compute_loss(outputs, observed_data)

# Modify and get another simulator
sim_reparam = (
    model
    .as_sim("baseline")
    .fix(p_vax=0.8, waning_half_life=365)
    .with_reparameterization(SEIR_R0_Reparam()) # Q = (R0, D)
    .with_transforms(R0="log", D="log")
    .build()
)
# sim_reparam: another ModelSimulator, now with Q coordinates
```

Listing 8: Fluent builder pattern for constructing packaged simulators. The `as_sim()` method starts a builder, fluent methods (`fix`, `with_transforms`, `with_reparameterization`) configure the coordinate system, and `build()` returns a `ModelSimulator`. Each operation returns another builder or simulator with the same call signature $(z, \text{seed}) \mapsto \text{outputs}$, demonstrating the closure property of the grammar.

Conceptually, `sim(z, seed)` implements the full downward chain

$$Z_V \xrightarrow{T_V^{-1}} P_V \xrightarrow{\kappa_V} \mathcal{M} \xrightarrow{\sigma} \mathcal{M} \times \mathcal{C} \xrightarrow{\text{Sim}(\cdot, s)} \mathcal{Y},$$

but the user only sees a simple callable that is compatible with any optimizer or sampler expecting a function "vector of reals + seed $\mapsto$ outputs".

After calibration, policy or counterfactual scenarios can be explored by applying their patches to draws from the posterior or to a point estimate in $\mathcal{M}$ and re-running the simulator. The key design constraint is that every grammatical operation (e.g., `as_sim`, `fix`, `with_transforms`, `with_reparameterization`, and similar future combinators) returns another simulator interface of the same overall shape. This closure property, analogous to how data manipulation verbs map data frames to data frames, allows users to compose operations freely while the library maintains the invariants described in the formal grammar.

Conclusion

This paper has proposed a grammar of model parameters for epidemiological simulation and calibration, built around a small number of spaces and maps: the full model parameter space $\mathcal{M}$, active free parameter views P_V , optional reparameterization spaces Q , and inference coordinate spaces Z_V . By treating common operations on model parameters (e.g. fixing, transforming, reparameterizing, and patching) as explicit maps between these spaces, we obtain a clean separation between the scientific content of a model and the mechanics of how parameters are exposed to calibration and workflow tooling.

At the level of model inputs, we argued for a principled partition into model parameters $\mathcal{M}$, configurations $\mathcal{C}$, and random seeds $\mathcal{S}$, and showed how scenarios can be represented as structured patches on $(\mathcal{M}, \mathcal{C})$ that are closed under composition. Within $\mathcal{M}$, parameter views V formalize the ubiquitous practice of holding some parameters fixed while varying others, via the projection $\pi_V : \mathcal{M} \rightarrow P_V$ and the embedding (fill map) $\kappa_V : P_V \rightarrow \mathcal{M}$. Optional reparameterizations $R : Q \rightarrow P_V$ and per-parameter transforms $T_V : P_V \rightarrow Z_V$ then provide a flexible but disciplined way to change coordinates for inference without modifying the underlying simulator $\text{Sim}(m, c, s)$ or altering the interpretation of parameters in $\mathcal{M}$.

For typical users, the primary value of this grammar is not that they will think explicitly in terms of $(\mathcal{M}, P_V, Q, Z_V)$, but that basing an implementation on this grammar supports robust, simple, and predictable front-door interfaces. A single calibration specification can encode a baseline parameter set, a scenario, the subset of parameters that are free to vary, and simple hints about transforms (e.g. “optimize on a log scale”), while the grammar ensures that these specifications are mapped correctly into simulator calls. Because the formalism cleanly separates scenarios from calibration views, and fixing from transforming from reparameterizing, it also enables precise validation and error messages when user specifications are inconsistent.

From an infrastructure and library-design perspective, the grammar provides a concrete contract and common substrate on which to build. Optimizers and samplers only need to concern themselves with working generically on $Z_V \cong \mathbb{R}^d$; different simulation backends can expose their parameters through the same $\mathcal{M}$ – P_V – Z_V interface; and complete calibration runs can be serialized in terms of points and traces in these spaces, improving reproducibility and auditability. The same structure supports programmatic introspection (listing parameters, views, transforms) and higher-level workflow tooling (e.g. automatic generation of grids, priors, etc.).

This grammar is not a complete calibration methodology: it does not address identifiability, the choice of priors or loss functions, or decision-theoretic objectives. Rather, it is intended as a foundational layer on which such choices can be expressed more cleanly. Future work could integrate this parameter grammar with a complementary “statistical grammar” of priors, likelihoods, and decision criteria, and explore richer reparameterizations and constraints for complex models. Even in its current form, however, the grammar provides a useful organizing principle for simulation software and workflows, helping to reduce ad-hoc design decisions and making routine but intricate parameter manipulations both more transparent and more reliable.

References

Buffalo, Vince (2015). *Bioinformatics Data Skills: Reproducible and Robust Research with Open Source Tools*. Sebastopol, CA: O’Reilly Media. ISBN: 978-1449367374.

- Gelman, Andrew et al. (2020). “Bayesian Workflow”. In: *Bayesian Analysis* 15.3, pp. 733–806. DOI: 10.1214/19-BA1195.
- McIlroy, M. Douglas (2019). “Unix pipes”. In: *The Unix System: A History and a Half*. Ed. by Diomidis Spinellis, Georgios Gousios, and Panagiotis Louridas. History of Programming Languages. Reprint and commentary on the development of Unix and pipes. ACM.
- Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, and Davis Vaughan (2023). *dplyr: A Grammar of Data Manipulation*. R package version 1.x. URL: <https://dplyr.tidyverse.org/>.
- Wilkinson, Leland (2005). *The Grammar of Graphics*. 2nd ed. Statistics and Computing. New York: Springer. ISBN: 978-0-387-24544-7.